Programación 1 — Unidad 10: Recursividad

Actividad 2: Ejercicios propuestos y eficiencia de la recursividad

Índice

– 10.3. Más ejemplos recursivos

– 10.4. Recursividad frente a iteración

– 10.5. Cuánto trabajo hace realmente una función recursiva

10.3. Más ejemplos recursivos

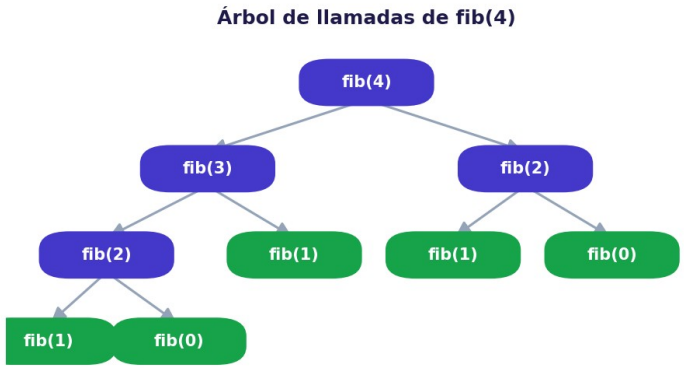
La recursividad brilla en problemas que se definen en términos de sí mismos. Veamos dos casos típicos: sumar los elementos de una lista y la sucesión de Fibonacci (donde cada número es la suma de los dos anteriores).

```
def suma(lista):
    if len(lista) == 0:
        return 0
    return lista[0] + suma(lista[1:])

print(suma([1, 2, 3, 4]))   # 10
```

Qué hace el código:

- `if len(lista) == 0: return 0` es el caso base: la suma de una lista vacía es 0.
- `lista[0] + suma(lista[1:])` suma el primer elemento más la suma del **resto** de la lista (sin el primero).
- `lista[1:]` es la lista sin su primer elemento: el problema se va achicando en cada llamada.
- Así, `suma([1,2,3,4])` es `1 + suma([2,3,4])`, que es `1 + (2 + suma([3,4]))`, y al final da 10.



El árbol de llamadas de fib(4): algunas llamadas se repiten, lo que muestra el costo de la recursión simple.

10.4. Recursividad frente a iteración

Muchos problemas se pueden resolver de las dos formas: con recursión o con un bucle (iteración). La recursión suele dar un código más corto y cercano a la definición matemática del problema, pero consume más memoria, porque cada llamada pendiente se acumula en una estructura llamada **pila de llamadas**.

Aspecto	Recursión	Iteración (bucles)
Legibilidad	Muy clara en problemas recursivos	Clara en repeticiones simples
Memoria	Mayor (acumula llamadas en la pila)	Menor
Ideal para	Estructuras anidadas, definiciones recursivas	Recorridos y conteos largos

Importante

Si una función recursiva no tiene un caso base correcto, o nunca lo alcanza, se produce una recursión infinita y Python corta el programa con un `RecursionError: maximum recursion depth exceeded`. Es el equivalente recursivo del bucle infinito: hay que asegurarse siempre de que el problema se achique y llegue al caso base.

Buenas prácticas

Usá recursividad cuando el problema es naturalmente recursivo (factoriales, recorridos de estructuras anidadas) y la cantidad de llamadas es razonable. Para repeticiones simples y muy largas, un bucle suele ser más eficiente y claro.

Encontrá el error

Este programa falla con un RecursionError. Fijate bien qué argumento recibe la llamada recursiva antes de seguir.

```
def cuenta_regresiva(n):
    if n == 0:
        print("Despegue!")
```

```
        return
    print(n)
    cuenta_regresiva(n)
```

Qué estaba mal

La llamada cuenta_regresiva(n) se llama a sí misma con el mismo n, en vez de n - 1. El problema nunca se achica, así que jamás se llega al caso base y la recursión es infinita.

Practicá la lectura

Explicá, paso a paso, cómo se van «apilando» las llamadas hasta llegar al caso base, y cómo se resuelven después en orden inverso.

```
def suma(lista):
    if not lista:
        return 0
    return lista[0] + suma(lista[1:])

print(suma([2, 4, 6]))
```

Para reflexionar y autoevaluarte

— ¿Qué te costó más de la recursividad: identificar el caso base o confiar en que la función «ya sabe resolver» el problema más chico?

— Pensando en la tabla de la sección 10.4, ¿en qué situación elegirías recursión aunque sepas que consume más memoria?

Antes de seguir, verificá que podés:

— Identificar el caso base y el caso recursivo de una función.

— Escribir una función recursiva que reduzca el problema en cada llamada.

— Explicar qué es la pila de llamadas.

— Comparar recursión e iteración y justificar cuándo conviene cada una.



Diagrama 10.4 generado con Mermaid.

10.5. Cuánto trabajo hace realmente una función recursiva

La tabla de la sección anterior compara recursión e iteración en memoria y legibilidad. Falta un ángulo más: cuántas veces se repite el mismo trabajo.

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Para calcular fibonacci(5), esta función recursiva termina llamando a fibonacci(2) tres veces distintas, y a fibonacci(1) y fibonacci(0) todavía más veces, recalculando una y otra vez los mismos valores.

Una versión con un bucle (o una recursiva que guarda los resultados ya calculados) computa cada valor **una sola vez**.

La lección no es «la recursión es mala»: es la misma idea de la sección 3.8 aplicada a funciones que se llaman a sí mismas. **Una solución más elegante no siempre es la más eficiente**, y vale la pena tenerlo presente al elegir entre recursión e iteración.